

Dự đoán nguy cơ đột quỵ bằng XGBoost, PCA và XAI

Tái hiện nghiên cứu Mochurad et al. (2025)

Học phần: HỌC MÁY VÀ KHAI PHÁ DỮ LIỆU

Giảng viên hướng dẫn: TRẦN TRỌNG HIẾU

Học viên thực hiện:

1. TÔNG VIỆT TÚ - Mã học viên: 25007845
2. NGUYỄN THỊ NHẬT LINH - Mã học viên: 25007871

Ngày 30 tháng 8 năm 2026

1. Bài toán và động cơ

- Mục tiêu: dự đoán sớm nguy cơ đột quỵ từ đặc trưng bệnh nhân.
- Khó khăn: **mất cân bằng lớp** (chỉ 4.3–4.9% ca dương tính), missing/outlier, quan hệ phi tuyến, yêu cầu giải thích.
- Hướng của bài báo: **XGBoost + PCA + SHAP**; PCA được tối ưu bằng OpenMP.

Ba tiêu chí chính

Độ chính xác + tốc độ xử lý + khả năng giải thích.

2. Hai bộ dữ liệu

Bộ dữ liệu	Quy mô	Đặc trưng chính
Dataset 1	5,110 bản ghi; 249 stroke (4.87%)	age, hypertension, heart disease, glucose, BMI, smoking, work type, gender, residence
Dataset 2	5,542,261 bản ghi; 235,961 stroke (4.26%)	Cấu trúc tương tự, dữ liệu tổng hợp quy mô lớn

- Chia 80/20 (stratified) cho cả hai tập; sau tiền xử lý (one-hot) đều có **21 đặc trưng đầu vào**.
- Dataset 1: cân bằng bằng **SMOTE**.
- Dataset 2: cân bằng bằng **random undersampling**.

3. Pipeline của nghiên cứu

- 1 Làm sạch, missing, outlier, one-hot encoding.
- 2 Chuẩn hóa dữ liệu.
- 3 Cân bằng lớp trên tập train (trong pipeline, tránh rò rỉ dữ liệu).
- 4 PCA giữ $\geq 95\%$ phương sai (Jacobi + OpenMP).
- 5 XGBoost + tuning hyperparameter (Grid Search, Stratified K-Fold).
- 6 Đánh giá: Accuracy, Precision, Recall, F1, ROC-AUC, MCC, Cohen's Kappa.
- 7 XAI bằng SHAP (Waterfall, Bar, Beeswarm).

4. PCA: giảm chiều

$$X_c = X - \bar{x}, \quad S = \frac{1}{N-1} X_c^T X_c,$$
$$S v_j = \lambda_j v_j, \quad X_{\text{reduced}} = X_c V_k.$$

- Chọn k sao cho cumulative explained variance đạt $\geq 95\%$.
- Theo bài báo: từ **21** đặc trưng sau tiền xử lý xuống còn **11** thành phần chính (cả hai tập dữ liệu).
- Phương sai giải thích đạt **96.90%** (Dataset 1) và **96.96%** (Dataset 2).
- Jacobi rotation được song song hóa bằng OpenMP.

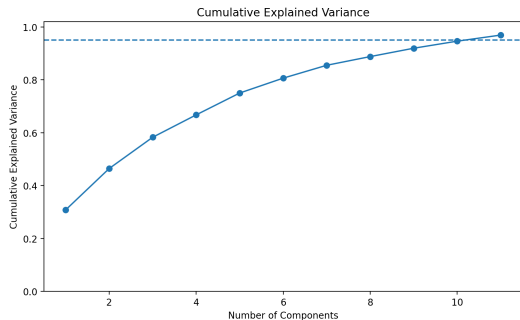
5. XGBoost và hàm mục tiêu

$$L(\theta) = \sum_{i=1}^N \ell(y_i, \hat{y}_i(\theta)) + \Omega(\theta)$$

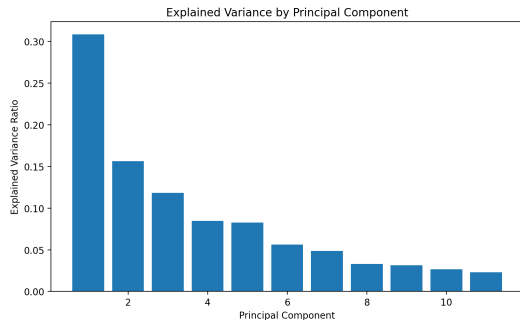
$$\ell(y_i, \hat{y}_i) = -[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)].$$

- Tuning: depth, learning rate, số cây, gamma, min child weight, subsample, colsample.
- Regularization giúp kiểm soát overfitting.
- ROC-AUC là tiêu chí quan trọng khi lựa chọn mô hình (Grid Search + Stratified K-Fold).

6.1. Kết quả PCA (Dataset 1)

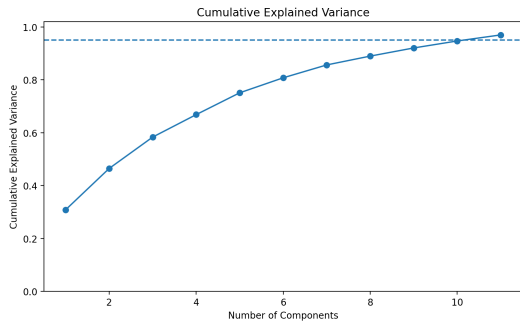


Phương sai tích lũy theo thành phần

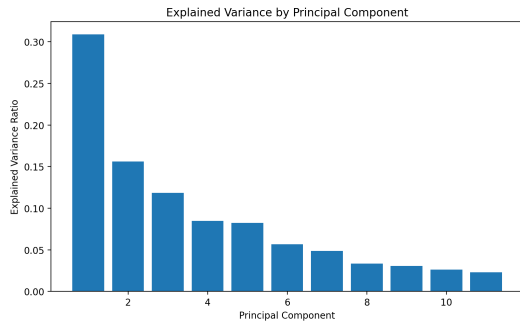


Tỷ lệ phương sai từng thành phần

6.2. Kết quả PCA (Dataset 2)



Phương sai tích lũy theo thành phần



Tỷ lệ phương sai từng thành phần

7. Hiệu năng mô hình baseline trên tập kiểm tra

- Mô hình đạt **Accuracy** cao trên cả 4 tập dữ liệu (0.9491–0.9638).
- Tuy nhiên, **Recall lớp 1 thấp**, chỉ từ 0.0400 đến 0.1538.
- Nguyên nhân: do tỷ lệ lớp dương thấp (4.3-4.9
- **Nhận xét:** Để tăng khả năng dự báo,
 - Ngoài Accuracy, cần bổ sung Precision, Recall, F1-score hay AUC-ROC
 - Nên áp dụng các kỹ thuật cân bằng dữ liệu + tinh chỉnh siêu tham số.

Chỉ tiêu	D1– không PCA	D1– có PCA	D2– không PCA	D2– có PCA
Accuracy	0.9501	0.9491	0.9612	0.9638
Precision (lớp 1)	0.4286	0.3333	0.9152	0.9723
Recall (lớp 1)	0.0600	0.0400	0.0986	0.1538
F1 (lớp 1)	0.1053	0.0714	0.1780	0.2656
ROC-AUC	0.8307	0.8011	0.9319	0.9641
MCC	0.1462	0.1013	0.2934	0.3792

Hiệu năng mô hình baseline trên tập kiểm tra

8. Kết quả tinh chỉnh siêu tham số (Grid Search)

Bảng: Siêu tham số tốt nhất tìm được bằng Grid Search

Siêu tham số	Dataset 1	Dataset 2
n_estimators	200	400
max_depth	3	5
learning_rate	0.03	0.10
min_child_weight	5	1
gamma	0.2	0.0
subsample	0.8	0.8
colsample_bytree	0.8	0.8
ROC-AUC tốt nhất (CV tuning)	0.8028	0.9913

9. Kết quả kiểm định chéo (Cross-validation)

Bảng: Kết quả kiểm định chéo 10-fold (trung bình $\pm$ độ lệch chuẩn)

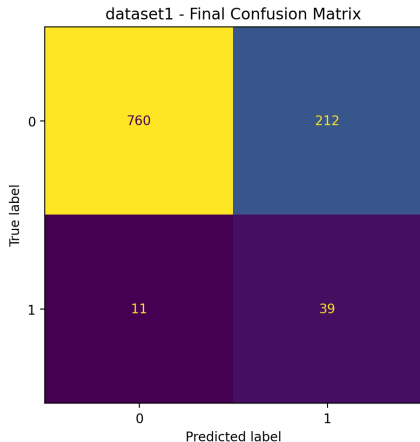
Chỉ số	Dataset 1	Dataset 2
Accuracy	0.7652 ± 0.0308	0.9654 ± 0.0010
Precision	0.1420 ± 0.0302	0.5519 ± 0.0073
Recall	0.7426 ± 0.1065	0.9937 ± 0.0005
F1-score	0.2381 ± 0.0472	0.7097 ± 0.0060
ROC-AUC	0.8238 ± 0.0425	0.9975 ± 0.0002
Thời gian CV (giây)	4.59	93.87

10. Kết quả mô hình cuối cùng trên tập kiểm tra

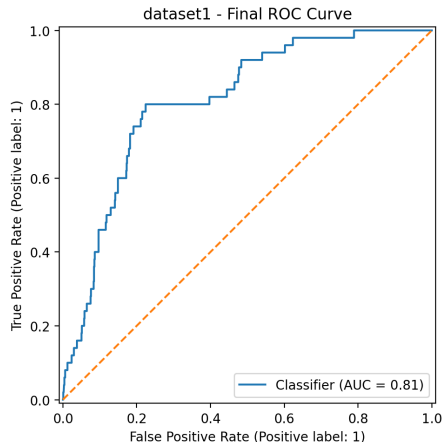
Bảng: Hiệu năng trên tập kiểm tra (hold-out cuối cùng)

Chỉ số	Dataset 1 ($n = 1,022$)	Dataset 2 ($n = 1,108,453$)
Accuracy	0.7818	0.9664
Precision (lớp 1)	0.1554	0.5590
Recall (lớp 1)	0.7800	0.9944
F1 (lớp 1)	0.2591	0.7157
ROC-AUC	0.8118	0.9976
MCC	0.2816	0.7322
Cohen's Kappa	0.1933	0.6993
TN / FP / FN / TP	760 / 212 / 11 / 39	1,024,239 / 37,022 / 263 / 46,929

10.1. Kết quả mô hình cuối cùng trên tập kiểm tra - Dataset 1

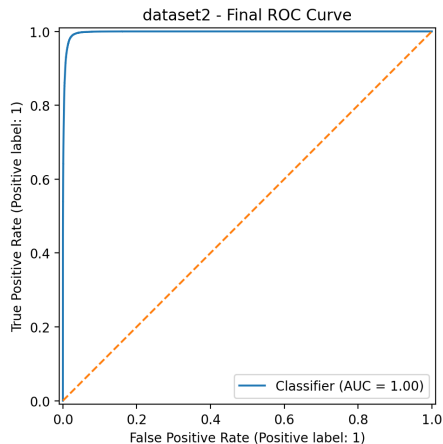
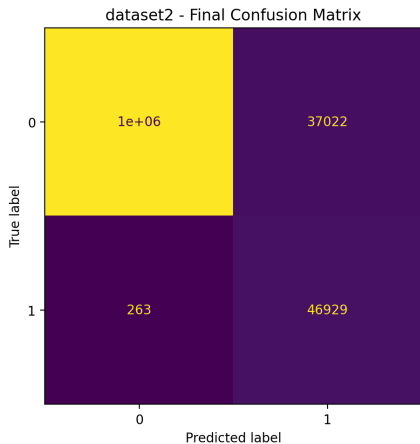


Hình: Ma trận nhầm lẫn của mô hình cuối cùng - Dataset 1



Hình: Đường cong ROC-AUC của mô hình cuối cùng - Dataset 1

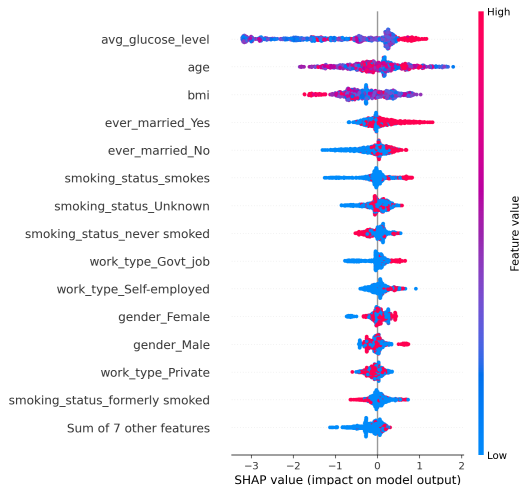
10.2. Kết quả mô hình cuối cùng trên tập kiểm tra - Dataset 2



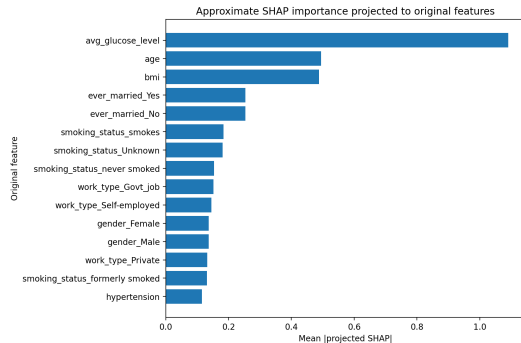
Hình: Ma trận nhầm lẫn của mô hình cuối cùng - Dataset 2

Hình: Đường cong ROC-AUC của mô hình cuối cùng - Dataset 2

11. SHAP: mô hình dự đoán dựa vào đâu? - Dataset 1



SHAP Beewarm – Dataset 1

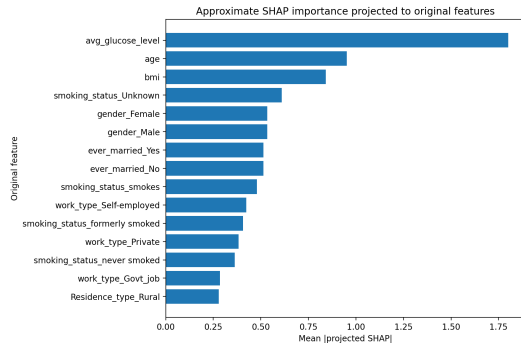
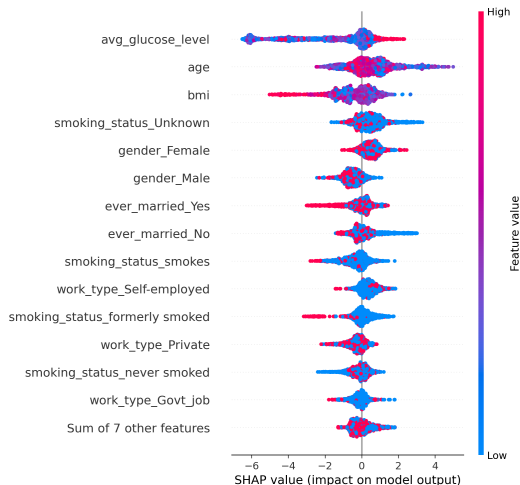


SHAP Bar – Dataset 1

Nhận xét

Mức glucose máu trung bình, tuổi, BMI có ảnh hưởng lớn nhất đến dự báo.

11. SHAP: mô hình dự đoán dựa vào đâu? - Dataset 2



SHAP Bar – Dataset 2

Nhận xét

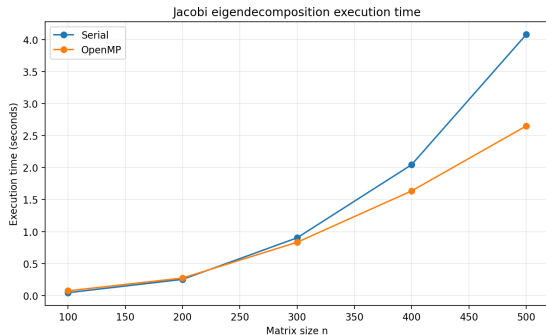
Mức glucose máu trung bình, tuổi, BMI có ảnh hưởng lớn nhất đến dự báo.

12. OpenMP và tốc độ (thuật toán Jacobi)

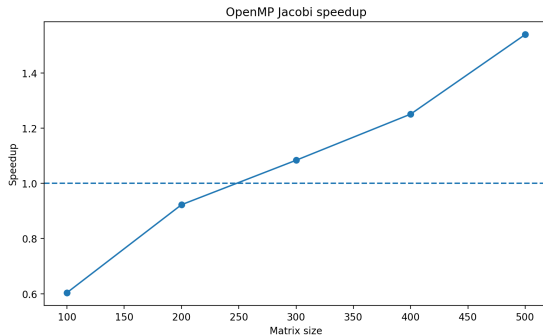
Kích thước ma trận N	100	200	300	400	500
Tuần tự (s)	0,0441	0,2521	0,9019	2,0456	4,0797
OpenMP (s)	0,0731	0,2733	0,8323	1,6355	2,6504
Speedup	0,604	0,923	1,084	1,251	1,539

- $N = 100$ và $N = 200$: OpenMP **chưa** có lợi thế (overhead song song hóa lớn hơn lợi ích).
- Từ $N = 300$ trở đi: OpenMP bắt đầu nhanh hơn, khoảng cách tăng dần theo kích thước ma trận.
- Kết luận: song song hóa chỉ có lợi khi kích thước bài toán đủ lớn, không phải lúc nào cũng nhanh hơn.

12. OpenMP và tốc độ (thuật toán Jacobi)



Thời gian thực thi Jacobi theo kích thước ma trận



Speedup của OpenMP so với phiên bản tuần tự

13. Thiết kế code tái hiện

- Train/test = 80/20, stratified theo biến mục tiêu.
- IQR bounds học từ train rồi áp cho test (tránh rò rỉ dữ liệu).
- Sampler (SMOTE/undersampling) và PCA nằm trong `imblearn.Pipeline`, chỉ fit trên phần train của mỗi fold.
- So sánh: baseline (không PCA), sau cân bằng lớp + tinh chỉnh siêu tham số, có/không PCA.
- Đánh giá hai bước: kiểm định chéo 10-fold, sau đó một lần hold-out cuối cùng.
- Xuất tự động: metrics, confusion matrix, ROC, PCA (cumulative variance, loadings), SHAP.

14. Kết luận

- XGBoost + PCA + XAI là pipeline hợp lý cho bài toán dự đoán đột quy.
- PCA cải thiện có ý nghĩa thống kê trên Dataset 2 (dữ liệu lớn) nhưng lại làm giảm nhẹ accuracy trên Dataset 1 (dữ liệu nhỏ) – lợi ích phụ thuộc quy mô dữ liệu.
- Accuracy cao có thể gây hiểu lầm trên dữ liệu mất cân bằng mạnh; cần xem cùng Precision, Recall, ROC-AUC, MCC.
- SHAP giúp diễn giải (glucose, tuổi, BMI là yếu tố nổi bật) nhưng không thay thế kiểm định lâm sàng.
- Mục tiêu thực nghiệm: **tái hiện và đối chiếu**, không giả định sẽ khớp tuyệt đối số liệu bài báo gốc.